



UT1. MANEJO DE FICHEROS

ACTIVIDAD 5

Acceso aleatorio a un archivo

Título: Acceso aleatorio a un archivo utilizando RandomAccessFile

Descripción:

En esta actividad vamos a trabajar con un archivo que simula un **registro de usuarios de un sistema de seguridad**. El objetivo es practicar el uso de la clase RandomAccessFile para realizar operaciones de lectura y escritura en posiciones concretas del archivo.

Aprenderás a moverte con el puntero del archivo para consultar un usuario específico, modificar sus datos, añadir nuevos usuarios y simular la eliminación de otros. Estas operaciones son frecuentes en la gestión de **bases de datos de credenciales, auditorías o registros de acceso**, donde se necesita modificar solo una parte de la información sin reescribir todo el fichero.

Objetivos:

- Comprender cómo funciona el acceso aleatorio a un archivo en Java.
- Aprender a utilizar los métodos seek(), read(), y write() de la clase RandomAccessFile.
- Manipular el puntero de archivo para leer y escribir datos en posiciones específicas.

Instrucciones:

1. Crea una nueva clase llamada **AccesoUsuarios** dentro del proyecto.
2. Desde el propio programa (no a mano con un editor), crea en el directorio del proyecto un archivo llamado usuarios.dat con al menos cinco registros de longitud fija. Cada registro debe ocupar exactamente 15 bytes:
 - ID de usuario: 3 cifras (001, 002...).
 - Un espacio.
 - Nombre: 10 caracteres. Si el nombre es más corto, se rellena con espacios a la derecha.
 - Salto de línea \n (1 byte).

Ejemplo del registro del usuario 003: **003 Mila** \n Usa nombres sin tildes ni eñes: 001 Juan, 002 Maria, 003 Mila, 004 Abraham, 005 Carlos.

3. En el método main(), realiza las siguientes operaciones:
 - **Lectura aleatoria:** Usa el método seek() para mover el puntero al registro del usuario número 003 y lee su nombre. Muestra el resultado en la consola.



- **Modificación de datos:** Mueve el puntero al registro del usuario con ID 002 y cambia su nombre por "Pedro". Vuelve a mostrar todo el contenido del archivo en la consola para confirmar el cambio.
 - **Añadir un nuevo registro:** Añade un nuevo usuario (006 Ana) al final del archivo utilizando el método seek() para situar el puntero en la posición correcta.
 - **Eliminar un registro:** (Opcional) Mueve el puntero al registro del usuario 004 y sobrescribe sus datos con ceros o espacios para simular la eliminación. Muestra el contenido final del archivo en consola.
4. Cierra el archivo correctamente después de cada operación.

Rúbrica:

0	No se cumple ninguno de los requisitos mínimos.
3	Presenta la tarea, pero el código no es correcto.
5	Se presenta la tarea y el código es correcto, pero no se demuestra su ejecución con evidencias (capturas o logs de consola).
7	Se presenta la tarea, el código es correcto, y se demuestra el funcionamiento mediante capturas o salida de consola; sin embargo, no se describen los principales métodos y clases del programa.
9	Se completa la tarea correctamente en todos sus apartados, con código funcional, capturas de consola y breve descripción de métodos y clases.
10	Se añade alguna funcionalidad extra o detalle innovador que mejore la aplicación (por ejemplo, mostrar estadísticas de cuántos usuarios activos hay, añadir marcas temporales al registrar o modificar un usuario).

Entrega:

1. Incluye aquí el código fuente de la clase desarrollada (AccesoUsuarios.java):

```
import java.io.File;
import java.io.IOException;
import java.io.RandomAccessFile;

/**
 * @author Juan Gabriel Galarza Claros
 */
public class AccesoUsuarios {
    // Nombre del archivo de datos
    private static final String ARCHIVO = "usuarios.dat";

    // Bytes reservados para el nombre
    private static final int TAM_NOMBRE = 10;

    // Bytes que ocupa cada registro: ID (3) + espacio (1) + nombre (10) + salto (1) = 15 bytes.
    private static final int TAM_REGISTRO = 3 + 1 + TAM_NOMBRE + 1;

    public static void main(String[] args) throws IOException {
        // 0) Crear y añadir los usuarios iniciales
    }
```



```
crearFichero();

// 1) Mostrar contenido inicial
System.out.println("--- CONTENIDO INICIAL DEL ARCHIVO ---");
mostrarFichero();

// 2) Lectura aleatoria: Leamos el usuario número 3
System.out.println("\n--- LECTURA ALEATORIA ---");
leerUsuario(3);

// 3) Modificación de datos: Cambiar el nombre del usuario ID 2 a "Pedro"
System.out.println("\n--- MODIFICACIÓN DE DATOS ---");
modificarNombre(2, "Pedro");
mostrarFichero();

// 4) Añadir un nuevo registro: Añadir usuario (006 Ana) al final
System.out.println("\n--- AÑADIR REGISTRO AL FINAL ---");
añadirUsuario(6, "Ana");
mostrarFichero();

// 5) Eliminar un registro: Sobrescribir el usuario ID 4 con espacios
System.out.println("\n--- ELIMINACIÓN DE REGISTRO ---");
eliminarUsuario(4);
mostrarFichero();
}

/**
 * Formatea un registro a exactamente 15 bytes según la estructura:
 * %03d (ID 3 dígitos) + " " + %-10s (Nombre relleno a la derecha) + "\n"
 */
private static String formatearRegistro(int id, String nombre) {
    if (nombre.length() > TAM_NOMBRE) {
        nombre = nombre.substring(0, TAM_NOMBRE);
    }
    return String.format("%03d %-10s\n", id, nombre);
}

/**
 * Crea el fichero 'usuarios.dat' con los 5 usuarios iniciales.
 */
public static void crearFichero() throws IOException {
    File f = new File(ARCHIVO);
    if (f.exists()) {
        f.delete();
    }

    try (RandomAccessFile raf = new RandomAccessFile(ARCHIVO, "rw")) {
        raf.writeBytes(formatearRegistro(1, "Juan"));
        raf.writeBytes(formatearRegistro(2, "Maria"));
        raf.writeBytes(formatearRegistro(3, "Mila"));
        raf.writeBytes(formatearRegistro(4, "Abraham"));
        raf.writeBytes(formatearRegistro(5, "Carlos"));
    }
}

/**
```



```
/* Lee un usuario específico en base a su posición lógica (1-based index).
*/
public static void leerUsuario(int posicion) throws IOException {
    try (RandomAccessFile raf = new RandomAccessFile(ARCHIVO, "r")) {
        long bytePos = (long) (posicion - 1) * TAM_REGISTRO;

        if (bytePos >= raf.length()) {
            System.out.println("El registro especificado no existe.");
            return;
        }

        raf.seek(bytePos);
        String linea = raf.readLine();

        if (linea != null && linea.length() >= 14) {
            String id = linea.substring(0, 3);
            String nombre = linea.substring(4, 14).trim();
            System.out.println("Usuario en posición " + posicion + " [ID: " + id + "] -> Nombre: " +
nombre);
        }
    }
}

/**
 * Modifica el nombre de un usuario por su posición/ID.
 */
public static void modificarNombre(int posicion, String nuevoNombre) throws IOException {
    try (RandomAccessFile raf = new RandomAccessFile(ARCHIVO, "rw")) {
        long bytePos = (long) (posicion - 1) * TAM_REGISTRO;

        if (bytePos >= raf.length()) {
            System.out.println("No se puede modificar: El registro no existe.");
            return;
        }

        raf.seek(bytePos);
        raf.writeBytes(formatearRegistro(posicion, nuevoNombre));
        System.out.println("Nombre del usuario " + posicion + " cambiado a: " + nuevoNombre);
    }
}

/**
 * Añade un nuevo usuario posicionándose directamente al final del fichero con seek().
 */
public static void añadirUsuario(int id, String nombre) throws IOException {
    try (RandomAccessFile raf = new RandomAccessFile(ARCHIVO, "rw")) {
        raf.seek(raf.length());
        raf.writeBytes(formatearRegistro(id, nombre));
        System.out.println("Usuario 00" + id + " (" + nombre + ") añadido al final.");
    }
}

/**
 * Simula la eliminación de un usuario sobrescribiendo su registro con espacios.
 */
public static void eliminarUsuario(int posicion) throws IOException {
```



```
try (RandomAccessFile raf = new RandomAccessFile(ARCHIVO, "rw")) {
    long bytePos = (long) (posicion - 1) * TAM_REGISTRO;

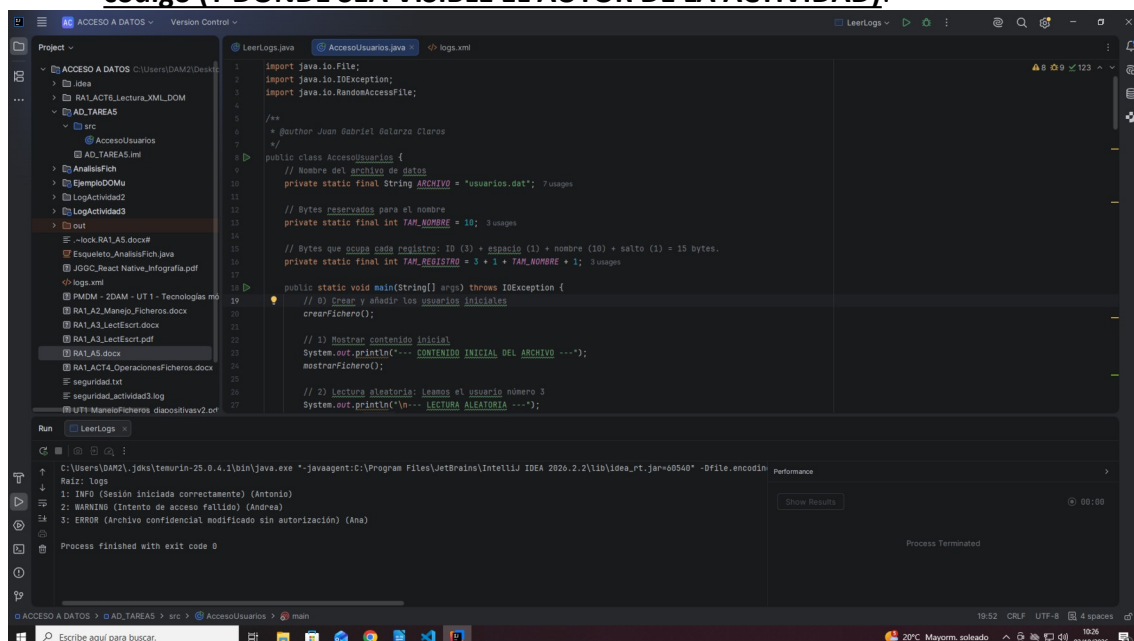
    if (bytePos >= raf.length()) {
        System.out.println("No se puede eliminar: Registro inexistente.");
        return;
    }

    raf.seek(bytePos);
    String borrado = "          \n"; // 14 espacios + \n
    raf.writeBytes(borrado);
    System.out.println("Registro en la posición " + posicion + " eliminado (sobrescrito).");
}
}

/*
 * Muestra todo el contenido actual del archivo por consola.
 */

public static void mostrarFichero() throws IOException {
    try (RandomAccessFile raf = new RandomAccessFile(ARCHIVO, "r")) {
        raf.seek(0);
        String linea;
        int numLinea = 1;
        while ((linea = raf.readLine()) != null) {
            System.out.println("Reg " + numLinea + ": [" + linea + "]");
            numLinea++;
        }
    }
}
```

2. Incluye aquí captura/s de pantalla que demuestren la correcta ejecución del código (Y DONDE SEA VISIBLE EL AUTOR DE LA ACTIVIDAD):





3. Describe aquí las clases, métodos y variables principales utilizadas en el programa:

Clase Principal

- **AccesoUsuarios**: Clase contenedora que gestiona la creación, lectura, modificación y eliminación de usuarios mediante registros de longitud fija en un archivo binario.

Variables Principales

- **ARCHIVO** (String): Ruta y nombre del fichero físico (usuarios.dat).
- **TAM_REGISTRO** (int): Constante con el tamaño fijo de cada registro (**15 bytes**).
- **TAM_NOMBRE** (int): Longitud máxima reservada para el nombre (**10 bytes**).

Métodos Principales

- **main(String[] args)**: Punto de entrada que ejecuta secuencialmente el flujo de operaciones requeridas.
- **crearFichero()**: Inicializa el archivo usuarios.dat escribiendo los 5 registros base formateados.
- **leerUsuario(int id)**: Realiza una **lectura aleatoria** desplazando el puntero con seek() hasta el registro deseado ((id - 1) * 15) e imprimiéndolo.
- **modificarNombre(int id, String nuevoNombre)**: Posiciona el puntero en el registro objetivo mediante seek() y sobrescribe sus datos con el nuevo nombre.
- **añadirUsuario(int id, String nombre)**: Mueve el puntero al final del archivo mediante seek(raf.length()) para adjuntar un nuevo registro.
- **eliminarUsuario(int id)**: Simula el borrado físico de un usuario posicionándose con seek() y sobrescribiendo su espacio con 14 espacios y un salto de línea (\n).